Lab Assignment 1:
Algorithm Foundations

Name: Vidit Bansal

Roll No: 2401420018

Course: B.Tech CSE (Data Science) IBM

Section: A

Course Name: Analysis and Design of Algorithms Lab (ENCS256)

Submitted to-

Dr. Aarti Sangwan

Signature

Task 1: Algorithm Growth Observation

Objective

To implement and visualise the growth of different time complexities.

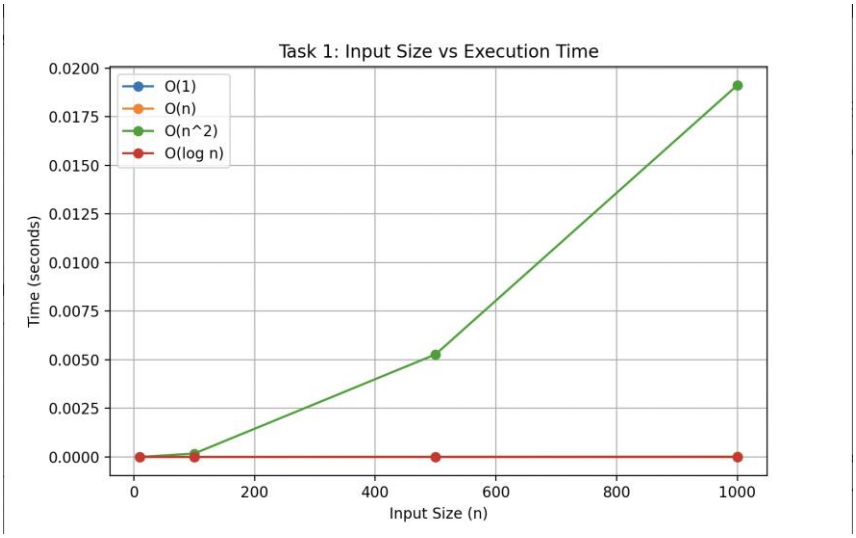
Code Input:

```python
import time
import matplotlib.pyplot as plt

# 1. Constant Time: O(1)
def constant_algo(n):
    return 1  # Always one step

# 2. Linear Time: O(n)
def linear_algo(n):
    count = 0
    for i in range(n):
        count += 1
    return count

# 3. Quadratic Time: O(n^2)
def quadratic_algo(n):
    count = 0
    for i in range(n):
        for j in range(n):
            count += 1
    return count

# 4. Logarithmic Time: O(log n)
def logarithmic_algo(n):
    count = 0
    while n > 1:
        n //= 2
        count += 1
    return count

input_sizes = [10, 100, 500, 1000]
results = {"O(1)": [], "O(n)": [], "O(n^2)": [], "O(log n)": []}

# Measuring Time
for n in input_sizes:
    # Measure O(1)
    start = time.perf_counter()
    constant_algo(n)
    results["O(1)"].append(time.perf_counter() - start)

```

```python

    # Measure O(n)
    start = time.perf_counter()
    linear_algo(n)
    results["O(n)"].append(time.perf_counter() - start)

    # Measure O(n^2)
    start = time.perf_counter()
    quadratic_algo(n)
    results["O(n^2)"].append(time.perf_counter() - start)

    # Measure O(log n)
    start = time.perf_counter()
    logarithmic_algo(n)
    results["O(log n)"].append(time.perf_counter() - start)

# Plotting Input Size vs Execution Time
plt.figure(figsize=(8, 5))
for label, times in results.items():
    plt.plot(input_sizes, times, label=label, marker='o')

plt.title("Task 1: Input Size vs Execution Time")
plt.xlabel("Input Size (n)")
plt.ylabel("Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()
```

Code Results

- **O(1)**: Constant time.
- **O(n)**: Linear time.
- **O(n^2)**: Quadratic time.
- **O(log n)**: Logarithmic time.



Observations

1. **O(1)** and **O(log n)** lines are nearly flat, showing excellent efficiency.
2. **O(n)** shows a straight diagonal line.
3. **O(n^2)** curves sharply upwards, indicating poor performance for large data.

Task 2: Best, Average, and Worst-Case Analysis

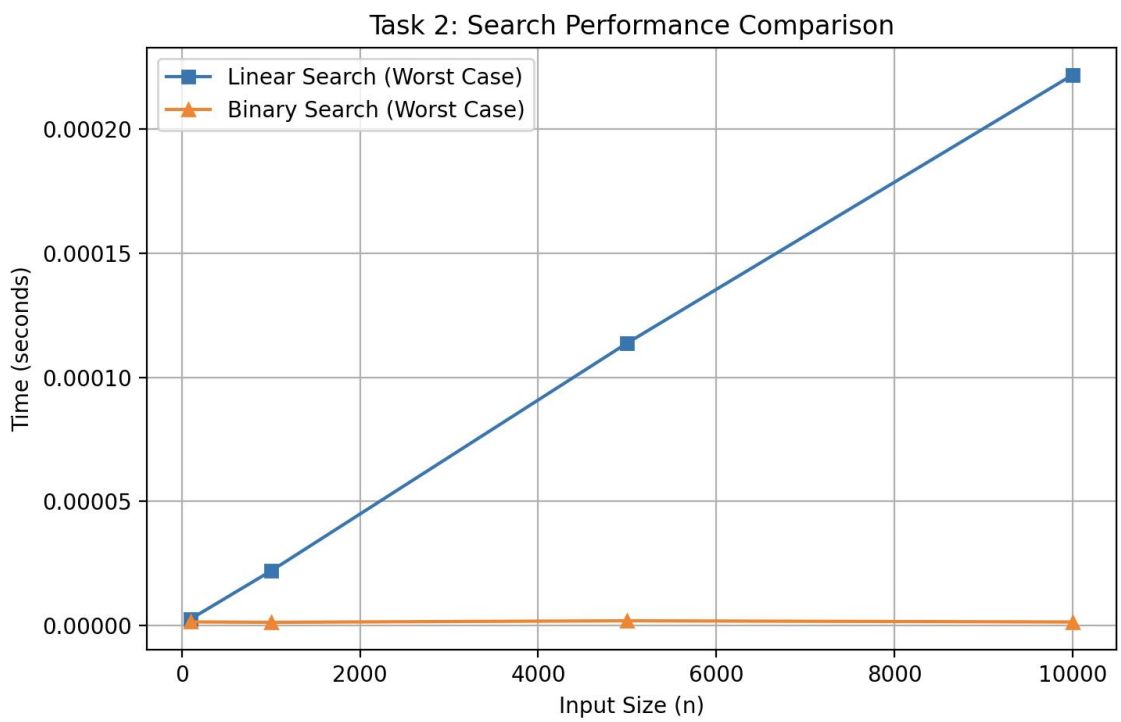
Objective

Measure the efficiency of Linear Search vs. Binary Search.

Code Input

```python
import time
import random
import matplotlib.pyplot as plt

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

sizes = [100, 1000, 5000, 10000]
ls_worst_times = []
bs_worst_times = []

for n in sizes:
    arr = sorted(random.sample(range(n * 10), n))
    target = -1

    start = time.perf_counter()
    linear_search(arr, target)
    ls_worst_times.append(time.perf_counter() - start)

    start = time.perf_counter()
    binary_search(arr, target)
    bs_worst_times.append(time.perf_counter() - start)

```

```python
# Comparison Plot
plt.figure(figsize=(8, 5))
plt.plot(sizes, ls_worst_times, label="Linear Search (Worst Case)", marker='s')
plt.plot(sizes, bs_worst_times, label="Binary Search (Worst Case)", marker='^')
plt.title("Task 2: Search Performance Comparison")
plt.xlabel("Input Size (n)")
plt.ylabel("Time (seconds)")
plt.legend()
plt.grid(True)
plt.show()
```



Observations

- **Linear Search**: Operates at O(n). If the element is at the end, it takes much longer as n increases.
- **Binary Search**: Operates at O(logn). It is vastly faster for large arrays because it halves the search area for each step.

Task 3: Recursion and Recurrence Validation

Objective

Compare basic recursion with optimised Dynamic Programming (DP).

```python
factorial_calls = 0

def factorial(n):
    global factorial_calls
    factorial_calls += 1

    if n == 0:
        return 1
    return n * factorial(n - 1)
```

✓ 0.0s

```python
fib_recursive_calls = 0

def fib_recursive(n):
    global fib_recursive_calls
    fib_recursive_calls += 1

    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)
```

✓ 0.0s

```python
def fib_dp(n):
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]
```

✓ 0.0s

```python
n_values = [5, 10, 20]

print("n | Factorial Calls | Fact Time | Fib Rec Calls | Fib Rec Time | Fib DP Time")
print("-"*80)

for n in n_values:
    # Factorial
    factorial_calls = 0
    start = time.time()
    factorial(n)
    fact_time = time.time() - start

    # Fibonacci Recursive
    fib_recursive_calls = 0
    start = time.time()
    fib_recursive(n)
    fib_rec_time = time.time() - start

    # Fibonacci DP
    start = time.time()
    fib_dp(n)
    fib_dp_time = time.time() - start

    print(f"{n} | {factorial_calls:^15} | {fact_time:.6f} | "
          f"{fib_recursive_calls:^14} | {fib_rec_time:.6f} | {fib_dp_time:.6f}")
```

✓ 0.0s

```python
# Factorial Time Graph
import matplotlib.pyplot as plt

n_values = list(range(1, 15))
factorial_calls = []

for n in n_values:
    factorial_calls.append(n + 1)

plt.figure()
plt.plot(n_values, factorial_calls)
plt.xlabel("n")
plt.ylabel("Number of Calls")
plt.title("Factorial Time Complexity")
plt.show()
```

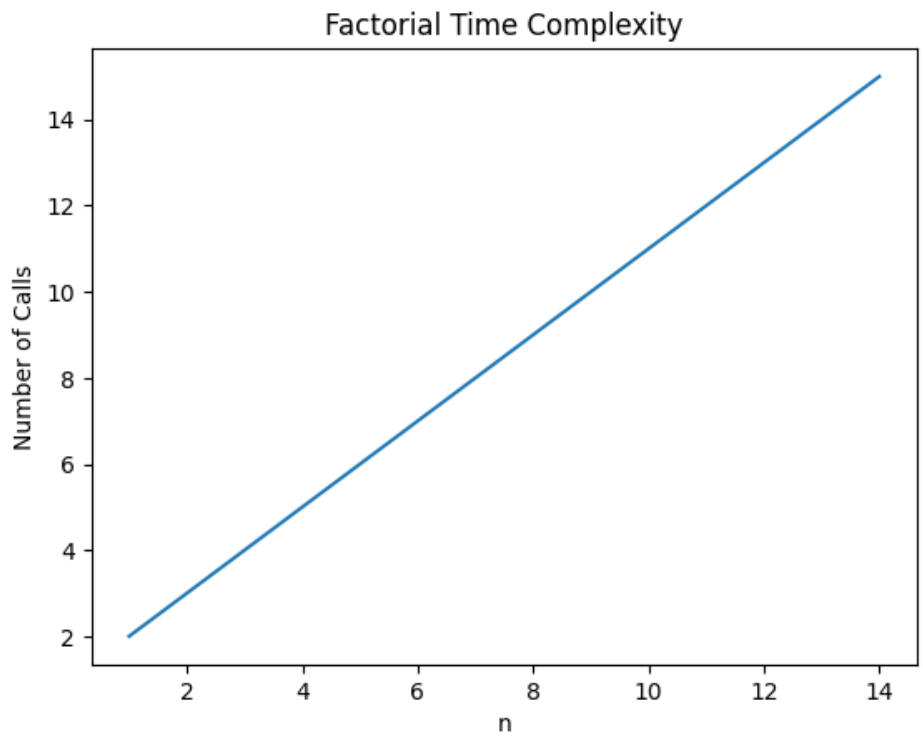
```python
# Fibonacci Recursive Time Graph

import matplotlib.pyplot as plt

def fib_calls(n):
    if n <= 1:
        return 1
    return fib_calls(n-1) + fib_calls(n-2) + 1

n_values = list(range(1, 15))
fib_call_count = []

for n in n_values:
    fib_call_count.append(fib_calls(n))

plt.figure()
plt.plot(n_values, fib_call_count)
plt.xlabel("n")
plt.ylabel("Number of Calls")
plt.title("Fibonacci Recursive Time Complexity")
plt.show()
```

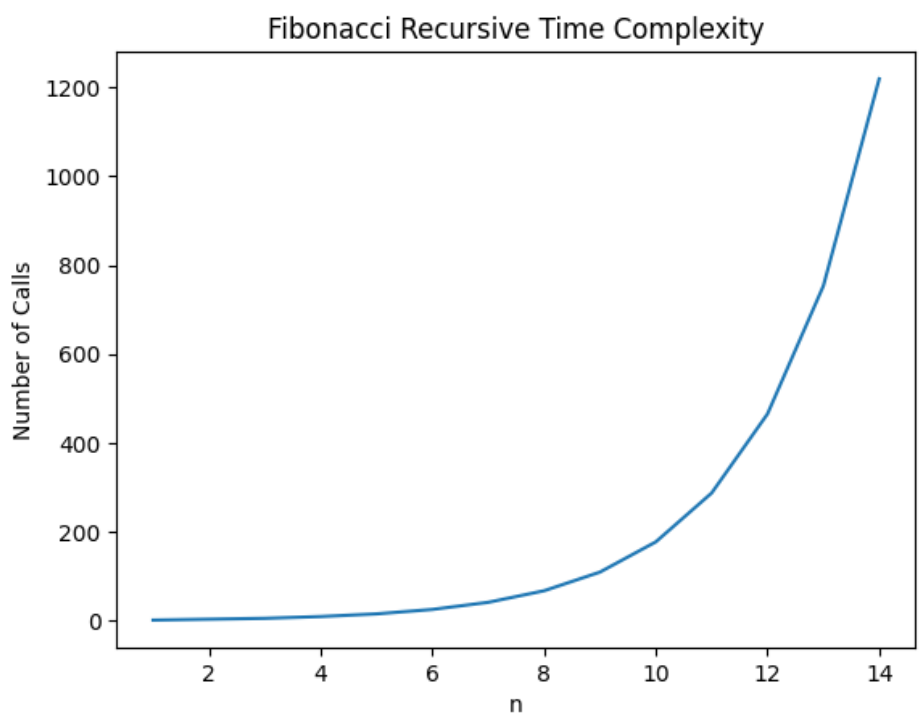
```python
# Fibonacci DP Time Graph
import matplotlib.pyplot as plt

n_values = list(range(1, 15))
dp_ops = []

for n in n_values:
    dp_ops.append(n)

plt.figure()
plt.plot(n_values, dp_ops)
plt.xlabel("n")
plt.ylabel("Operations")
plt.title("Fibonacci DP Time Complexity")
plt.show()
```

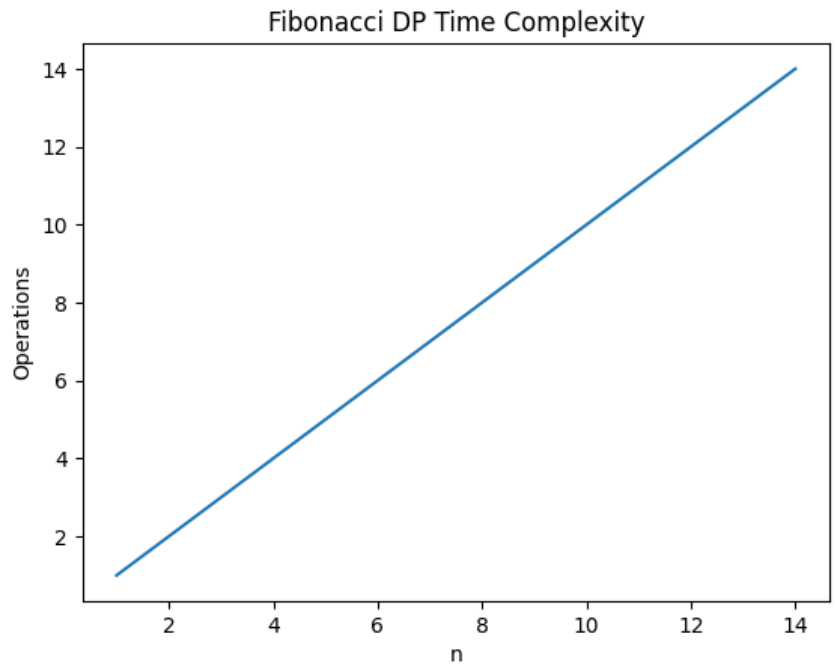
```python
# Space Complexity Graph
import matplotlib.pyplot as plt

n_values = list(range(1, 15))
space_used = []

for n in n_values:
    space_used.append(n)

plt.figure()
plt.plot(n_values, space_used)
plt.xlabel("n")
plt.ylabel("Space")
plt.title("Task 3 Space Complexity")
plt.show()
```

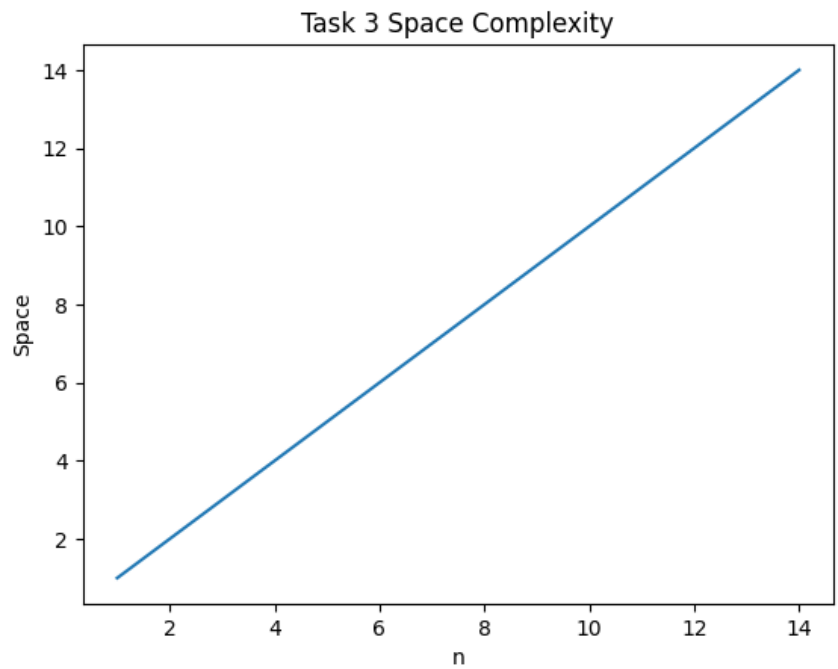
Code Output

```
[13]  ✓ 0.0s

...  n | Factorial Calls | Fact Time | Fib Rec Calls | Fib Rec Time | Fib DP Time
     ---------------------------------------------------------------------------
     5 |        6        | 0.000007 |       15       | 0.000006 | 0.000008
     10 |       11        | 0.000004 |      177       | 0.000037 | 0.000003
     20 |       21        | 0.000009 |     21891      | 0.005015 | 0.000018
```


Factorial Time Complexity
Number of Calls
n
2
4
6
8
10
12
14


Fibonacci Recursive Time Complexity
Number of Calls
n
0
200
400
600
800
1000
1200
2
4
6
8
10
12
14





Analysis

- The **Naive Fibonacci** approach recalculates the same values thousands of times (exponential growth).
- **DP Fibonacci** remembers previous answers, reducing the work to a simple linear O(n) process.

Task 4: Solving Recurrences through Code

Objective

Validate recursive call counts for standard recurrences.

Code Input

```python
import time

calls_rec1 = 0

def recurrence_1(n):
    global calls_rec1
    calls_rec1 += 1

    if n <= 1:
        return n
    return recurrence_1(n // 2) + n
```

4] ✓ 0.0s Python

```python
calls_rec2 = 0

def recurrence_2(n):
    global calls_rec2
    calls_rec2 += 1

    if n <= 1:
        return n
    return recurrence_2(n // 2) + recurrence_2(n // 2) + n
```

5] ✓ 0.0s Python

```python
# Recursion 1 GRAPH O(logn)

import matplotlib.pyplot as plt
import math

n_values = [10, 50, 100, 200, 500, 1000]
calls = []

for n in n_values:
    calls.append(math.log2(n))

plt.figure()
plt.plot(n_values, calls)
plt.xlabel("n")
plt.ylabel("Calls")
plt.title("Recurrence 1 Time Complexity")
plt.show()
```

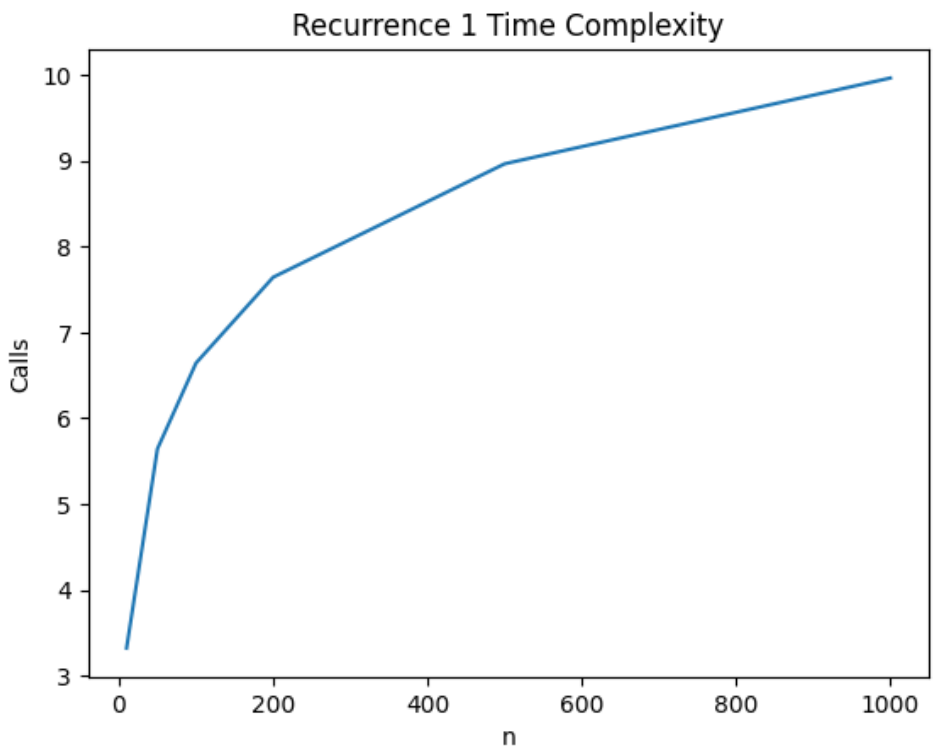
```python
n_values = [10, 100, 500, 1000]

print("n | Rec1 Calls | Rec1 Time | Rec2 Calls | Rec2 Time")
print("-" * 60)

for n in n_values:
    # Recurrence 1
    calls_rec1 = 0
    start = time.time()
    recurrence_1(n)
    time_rec1 = time.time() - start

    # Recurrence 2
    calls_rec2 = 0
    start = time.time()
    recurrence_2(n)
    time_rec2 = time.time() - start

    print(f"{n:<4}| {calls_rec1:^10} | {time_rec1:.6f} | "
          f"{calls_rec2:^10} | {time_rec2:.6f}")
```

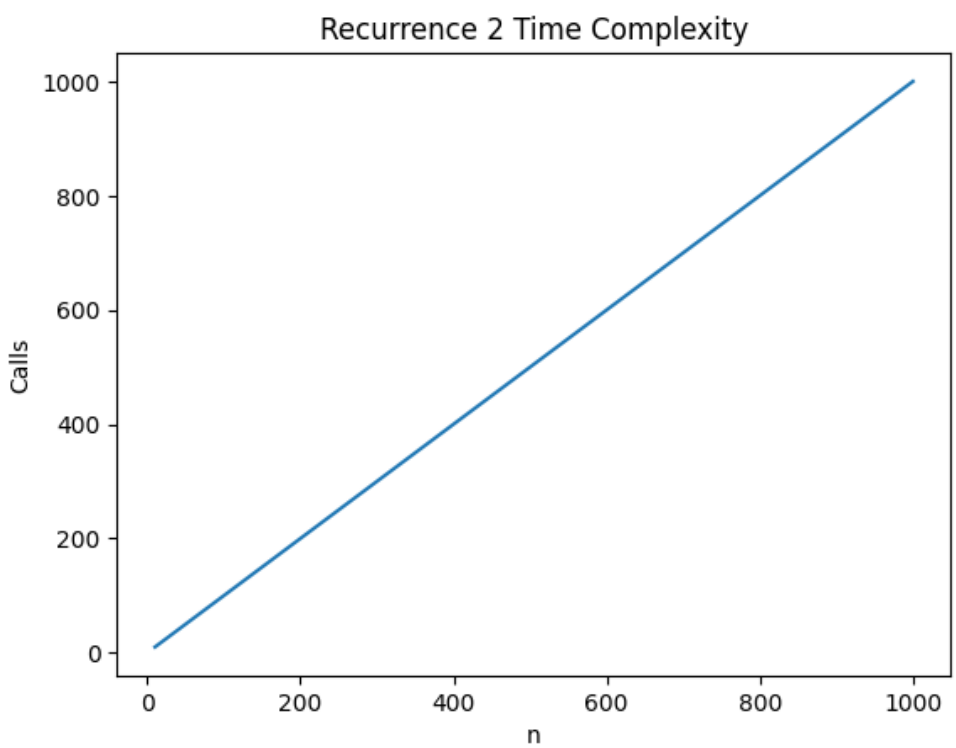
✓ 0.0s

```python
# Recursion 2 Graph O(n)

import matplotlib.pyplot as plt

n_values = [10, 50, 100, 200, 500, 1000]
calls = []

for n in n_values:
    calls.append(n)

plt.figure()
plt.plot(n_values, calls)
plt.xlabel("n")
plt.ylabel("Calls")
plt.title("Recurrence 2 Time Complexity")
plt.show()
```

Code Output

```
n | Rec1 Calls | Rec1 Time | Rec2 Calls | Rec2 Time
-----------------------------------------------------------
10  |     4      | 0.000009 |     15     | 0.000011
100 |     7      | 0.000005 |    127     | 0.000041
500 |     9      | 0.000005 |    511     | 0.000164
1000|     10     | 0.000008 |    1023    | 0.000402
```





Observations

1. T(n) = T(n/2) + n: The calls increase slowly (logarithmic calls), matching O(n) total work.

2. $T(n) = 2T(n/2)$: The branching causes the call count to double at each level, matching the $O(\log n)$ complexity seen in Merge Sort.

Conclusion

This demonstrates that algorithm choice is more important than hardware speed. A quadratic algorithm becomes unusable quickly, while logarithmic or linear algorithms scale efficiently to handle real-world data sizes.

GitHub Link: https://github.com/vidit401/ADA-Lab-Assignment-1